

A Proposal to add stacktrace library

Significant changes since [P0881R2](#) are marked with blue.

☐ Show deleted lines from P0881R2.

I. Motivation

In the current working draft [\[N4741\]](#) there is no way to get, store and decode the current call sequence. Such call sequences are useful for debugging and post mortem debugging. They are popular in other programming languages (like Java, C#, Python).

Pretty often assertions can't describe the whole picture of a bug and do not provide enough information to locate the problem. For example, you can see the following message on out-of-range access:

```
boost/array.hpp:123: T& boost::array<T, N>::operator[](boost::array<T, N>::size_type): Assertion '(i < N)&&("out of range")' failed.  
Aborted (core dumped)
```

That's not enough information in the assert message to locate the problem without debugger.

This paper proposes classes that could simplify debugging and may change the assertion message into the following:

```
Expression 'i < N' is false in function 'T& boost::array<T, N>::operator[](boost::array<T, N>::size_type)': out of range.  
Backtrace:  
0# boost::assertion_failed_msg(char const*, char const*, char const*, char const*, long) at ../example/assert_handler.cpp:39  
1# boost::array<int, 5ul>::operator[](unsigned long) at ../../boost/array.hpp:124  
2# bar(int) at ../example/assert_handler.cpp:17  
3# foo(int) at ../example/assert_handler.cpp:25  
4# bar(int) at ../example/assert_handler.cpp:17  
5# foo(int) at ../example/assert_handler.cpp:25  
6# main at ../example/assert_handler.cpp:54  
7# 0x00007F991FD69F45 in /lib/x86_64-linux-gnu/libc.so.6  
8# 0x0000000000401139
```

II. Design Decisions

The design is based on the Boost.Stacktrace library, a popular library that does not depend on any non-standard library components.

Note about signal safety: this proposal does not attempt to provide a signal-safe solution for capturing and decoding stacktraces. Such functionality currently is not implementable on some of the popular platforms. However, the paper attempts to provide extensible solution, that may be made signal safe some day by providing a signal safe allocator and changing the stacktrace implementation details.

Note on performance: during Boost.Stacktrace development phase many users requested a fast way to store stacktrace, without decoding the function names. This functionality is preserved in the paper. All the `stack_frame` functions and constructors are lazy and won't decode the pointer information if there was no explicit request from class user.

Note on allocations: initial implementations of Boost.Stacktrace were not using allocator and all the frames were placed inside a fixed size internal storage. That was a mistake! Sometimes the most important information is located at the bottom of the stack. For example if you run Boost.Test, then the test name will be located low on the stack. With a fixed size storage the bottom of the stack could be lost along with the information.

Current design assumes that by default users wish to see the whole stack and OK with dynamic allocations, because do not construct stacktrace in performance critical places. For those users, who wish to use stacktrace on a hot path or in embedded environments `basic_stacktrace` allows to provide a custom allocator that allocates on the stack or in some other place, where users thinks it is appropriate.

Note on returning `std::string` and not having `noexcept` on `stack_frame::source_line()`: Unfortunately this is a necessity on some platforms, where getting source line requires allocating or where source file name [returned into](#) a storage [provided by user](#).

Note on expected implementation: We assume that Standard Library implementations would allow to disable/enable gathering stacktraces by a compiler switch that does not require recompiling the whole project. In other words, we expect to see a compiler option like `-fno-stacktrace` or `libstacktrace/lib_stacktrace_noop` libraries with the same ABI that would force the constructor of the `basic_stacktrace` to do nothing. This feature is implemented in Boost.Stacktrace and is highly requested in big projects.

Should stacktrace be a class or a function?

class	function
<pre>struct promise_type { std::vector<stack_frame> frames; };</pre>	<pre>struct promise_type { std::vector<stack_frame> frames; };</pre>

<pre> void append(const stacktrace& s) { frames.insert(frames.end(), s.begin(), s.end()); } void print() { for (int i=0; auto& frame: frames) { std::cout << i++ << " " << frame; } } }; </pre>	<pre> void append(const std::vector<stack_frame>& s) { frames.insert(frames.end(), s.begin(), s.end()); } void print() { std::cout << frames; } }; </pre>
<pre> class stacktrace { small_vector<stack_frame> frames_; platform-specific-cache-of-internals; public: operator bool() const noexcept; // almost all the vector interface }; </pre>	<pre> template<class Allocator = allocator<stack_frame>> vector<stack_frame, Allocator> stacktrace(const Allocator& alloc = Allocator{}) noexcept; template<class Allocator = allocator<stack_frame>> vector<stack_frame, Allocator> stacktrace(size_type skip, size_type max_depth, const Allocator& alloc = Allocator{}) noexcept; </pre>
<pre> template<class Allocator> string to_string(const basic_stacktrace<Allocator>& st); template<class charT, class traits, class Allocator> basic_ostream<charT, traits>& operator<< (basic_ostream<charT, traits>& os, const basic_stacktrace<Allocator>& st); </pre>	<pre> template<class Allocator> string to_string(const vector<stack_frame, Allocator>& st); template<class charT, class traits, class Allocator> basic_ostream<charT, traits>& operator<< (basic_ostream<charT, traits>& os, const vector<stack_frame, Allocator>& st); </pre>
<pre> stacktrace s; if (s) { std::cout << "backtrace: " << s; } </pre>	<pre> auto s = stacktrace(); if (!s.empty()) { std::cout << "backtrace: " << s; } </pre>

LEWG decided to leave it a separate type: "Prefer stacktrace as a type rather than `vector`." SF:0, F:3, N:5, A:0, SA:0

III. Significant changes to review

LEWG:

- Added hash support.
- Removed `stack_frame` constructors from function pointers and `stack_frame::native_ptr_t` alias.
- LWG requested `rbegin` like functions for `basic_stacktrace`, so now `basic_stacktrace` satisfies the requirements of an allocator-aware container, of a sequence container and reversible container except that only operations defined for const-qualified sequence containers are supported and that the semantics of comparison functions and default constructor are different from those required for a container.

LEWG was OK with the above changes.

SG16:

- `stack_frame::source_file()` and encodings.

SG16 discussed a number of options including the possibility of `source_file()` returning `std::filesystem::path`. SG16 converged on the following recommendation: "Align behavior with `source_location`; remove wording regarding conversion; string contents are implementation defined. ". No objection to unanimous consent.

CWG question to LEWG:

- `stack_frame::address()` function should return instruction pointer, or stack address (that may have contained the instruction pointer), or both?

LEWG in favour of instruction pointer: "`stack_frame::address()` should return (something like) the instruction pointer (only)." SF:4, F:7, N:0, A:0, SA:0.

Points of special interest for CWG:

- `stacktrace` definition in `[stacktrace.def]` and member functions in `[stack_frame.query]`. Wording should not prevent any optimizations.

IV. Wording Intent

Key features that should be preserved by implementations:

- All the functions are lazy and do not query the pointer information if there was no explicit request.
- No fixed max size for trace - all the available invokers must be stored in a `stacktrace`.
- Implementations should allow to disable/enable gathering stacktraces by a link-time switch.
- Stacktracing does not prevent any of the optimizations.
- No info for a pointer is OK (it's not an error, do not throw!).
- `stack_frame::description()` should return a demangled function signature if possible.

- `to_string(stacktrace)` should query information from debug symbols, symbol export tables and return demangled signatures if possible.

V. Wording

23.? Stacktrace

[stacktrace]

This subclause describes components that C++ programs may use to store the stacktrace of the current thread of execution and query information about the stored stacktrace at runtime.

23.?.1 Stacktrace definition

[stacktrace.def]

The *invocation sequence* of the current evaluation x_0 in the current thread of execution is a sequence $(x_0, \dots, x_n)$ of evaluations such that, for $i > 0$, x_i is within the function invocation x_{i-1} (6.8.1 [intro.execution]).

A *stacktrace* is an approximate representation of an invocation sequence and consists of *stack frames*, where each stack frame represents an evaluation.

23.?.2 Header <stacktrace> synopsis

[stacktrace.syn]

```
namespace std {
    // 23.?.3, class stack_frame
    class stack_frame;

    constexpr bool operator==(const stack_frame& x, const stack_frame& y) noexcept;
    constexpr bool operator!=(const stack_frame& x, const stack_frame& y) noexcept;
    constexpr bool operator<(const stack_frame& x, const stack_frame& y) noexcept;
    constexpr bool operator>(const stack_frame& x, const stack_frame& y) noexcept;
    constexpr bool operator<=(const stack_frame& x, const stack_frame& y) noexcept;
    constexpr bool operator>=(const stack_frame& x, const stack_frame& y) noexcept;

    // 23.?.4, class basic_stacktrace
    template<class Allocator>
    class basic_stacktrace;

    // basic_stacktrace typedef names
    using stacktrace = basic_stacktrace<allocator<stack_frame>>;

    // 23.?.5, non-member functions
    void swap(stack_frame& a, stack_frame& b) noexcept;

    template<class Allocator>
    void swap(basic_stacktrace<Allocator>& a, basic_stacktrace<Allocator>& b);

    template<class Allocator>
    string to_string(const basic_stacktrace<Allocator>& st);

    string to_string(const stack_frame& f);

    template<class charT, class traits, class Allocator>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& os, const basic_stacktrace<Allocator>& st);

    template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& os, const stack_frame& f);

    // 23.?.6, hash support
    template<class T> struct hash;
    template<> struct hash<stack_frame>;
    template<class Allocator> struct hash<basic_stacktrace<Allocator>>;
}
```

23.?.3 Class stack_frame

[stack_frame]

```
namespace std {
    class stack_frame {
    public:
        // 23.?.3.1, constructors
        constexpr stack_frame() noexcept;
        constexpr stack_frame(const stack_frame& other) noexcept = default;
        constexpr stack_frame& operator=(const stack_frame& other) noexcept = default;

        ~stack_frame() = default;

        // 23.?.3.2, observers
        constexpr const void* address() const noexcept;
        constexpr explicit operator bool() const noexcept;

        // 23.?.3.3, query
        string description() const;
        string source_file() const;
        uint_least32_t source_line() const;

        // 23.?.3.4, modifiers
        void swap(stack_frame& other) noexcept;
    };
}
```

```
};
}
```

An object of class `stack_frame` represents a stack frame and provides operations for querying information about it.

23.?.3.1 Construct

[stack_frame.cons]

```
constexpr stack_frame() noexcept;
```

Ensures: `address()` == nullptr.

23.?.3.2 Observers

[stack_frame.obs]

```
constexpr const void* address() const noexcept;
```

Returns: An unspecified representation of the evaluation represented by `*this`.

Remarks: Successive invocations of the `address()` function for the same `stack_frame` object return identical values.

```
constexpr explicit operator bool() const noexcept;
```

Returns: `address()` != nullptr.

23.?.3.3 Query

[stack_frame.query]

[Note: All the `stack_frame` query functions treat errors other than memory allocation errors as "no information available" and do not throw in that case. - end note]

```
string description() const;
```

Returns: A description of the evaluation represented by `*this`, if such information is available, or the empty string otherwise.

Throws: `bad_alloc` if memory for the internal data structures or the resulting string cannot be allocated.

```
string source_file() const;
```

Returns: The presumed or actual name of the source file [cpp.predefined] that lexically contains the expression or statement whose evaluation is represented by `*this`, if such information is available, or the empty string otherwise.

Throws: `bad_alloc` if memory for the internal data structures or the resulting string cannot be allocated.

```
uint_least32_t source_line() const;
```

Returns: A line number that lexically relates to the evaluation represented by `*this`. If `source_file` returns the presumed name of the source file, returns the presumed line number; if `source_file` returns the actual name of the source file, returns the actual line number. If such information is not available, returns 0.

Throws: `bad_alloc` if memory for the internal data structures cannot be allocated.

23.?.3.4 Modifiers

[stack_frame.mod]

```
void swap(stack_frame& other) noexcept;
```

Effects: Exchanges the contents of `*this` and `other`.

23.?.3.5 Comparison

[stack_frame.cmp]

```
constexpr bool operator==(const stack_frame& x, const stack_frame& y) noexcept;
```

Returns: `x.address() == y.address()`.

```
constexpr bool operator!=(const stack_frame& x, const stack_frame& y) noexcept;
```

Returns: `x.address() != y.address()`.

```
constexpr bool operator< (const stack_frame& x, const stack_frame& y) noexcept;
```

Returns: `less<>{}(x.address(), y.address())`.

```
constexpr bool operator> (const stack_frame& x, const stack_frame& y) noexcept;
```

Returns: `greater<>{}(x.address(), y.address())`.

```
constexpr bool operator<=(const stack_frame& x, const stack_frame& y) noexcept;
```

Returns: `less_equal<>{}(x.address(), y.address())`.

```
constexpr bool operator>=(const stack_frame& x, const stack_frame& y) noexcept;
```

Returns: greater_equal<>{}(x.address(), y.address()).

23.?.4 Class template basic_stacktrace

[stacktrace.basic.template]

```
namespace std {
    template<class Allocator>
    class basic_stacktrace {
    public:
        using value_type = stack_frame;
        using const_reference = const value_type&;
        using reference = value_type&;
        using const_iterator = implementation-defined;
        using iterator = const_iterator;
        using reverse_iterator = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;
        using difference_type = typename iterator_traits<iterator>::difference_type;
        using size_type = typename allocator_traits<Allocator>::size_type;
        using allocator_type = Allocator;

        // 23.?.4.1, construct/copy/destroy
        basic_stacktrace() noexcept;
        explicit basic_stacktrace(const allocator_type& alloc) noexcept;
        basic_stacktrace(size_type skip, size_type max_depth, const allocator_type& alloc = allocator_type()) noexcept;

        basic_stacktrace(const basic_stacktrace& other) = default;
        basic_stacktrace(basic_stacktrace&& other) noexcept = default;
        basic_stacktrace& operator=(const basic_stacktrace& other) = default;
        basic_stacktrace& operator=(basic_stacktrace&& other) = default;
        ~basic_stacktrace() = default;

        // 23.?.4.2, observers
        allocator_type get_allocator() const;

        const_iterator begin() const noexcept;
        const_iterator end() const noexcept;
        const_reverse_iterator rbegin() const noexcept;
        const_reverse_iterator rend() const noexcept;

        const_iterator cbegin() const noexcept;
        const_iterator cend() const noexcept;
        const_reverse_iterator crbegin() const noexcept;
        const_reverse_iterator crend() const noexcept;

        explicit operator bool() const noexcept;
        [[nodiscard]] bool empty() const noexcept;
        size_type size() const noexcept;
        size_type max_size() const noexcept;

        const_reference operator[](size_type ) const;
        const_reference at(size_type ) const;

        // 23.?.4.3, comparisons
        template <class Allocator2>
        strong_ordering operator<=>(const basic_stacktrace< Allocator2 >& rhs) const noexcept;

        // 23.?.4.4, modifiers
        void swap(basic_stacktrace& other);

    private:
        vector<value_type, allocator_type> frames; // exposition only
    };
}
```

The basic_stacktrace template class stores the stacktrace of the current thread of execution on construction and provides access to the stored stacktrace.

The class template basic_stacktrace satisfies the requirements of an allocator-aware container, of a sequence container and reversible container (21.2.1, 21.2.3) except that only operations defined for const-qualified sequence containers are supported and that the semantics of comparison functions and default constructor are different from those required for a container.

23.?.4.1 Construct/copy/destroy

[stacktrace.basic.cons]

```
basic_stacktrace() noexcept;
explicit basic_stacktrace(const allocator_type& alloc) noexcept;
```

Effects: Stores the stacktrace of the current evaluation in the current thread of execution in frames. alloc is passed to the frames constructor.

[Note: If the stacktrace was successfully obtained, then frames.front() is the stack_frame representing approximately the current evaluation, and frames.back() is the stack_frame representing approximately the initial function of the current thread of execution. - end note]

Ensures: !*this if failed to store stacktrace of the current thread of execution; !!*this otherwise.

```
basic_stacktrace(size_type skip, size_type max_depth, const allocator_type& alloc = allocator_type()) noexcept;
```

Let t be a stacktrace as-if obtained via basic_stacktrace(alloc). Let n be t.size().

Ensures: frames contains the values [t.begin() + min(n, skip), t.begin() + min(n, skip + max_depth)).

23.?.4.2 Observers

[stacktrace.basic.obs]

allocator_type get_allocator() const;

Returns: frames.get_allocator().

const_iterator begin() const noexcept;
const_iterator cbegin() const noexcept;

Returns: frames.cbegin().

const_iterator end() const noexcept;
const_iterator cend() const noexcept;

Returns: frames.cend().

const_iterator rbegin() const noexcept;
const_iterator crbegin() const noexcept;

Returns: frames.crbegin().

const_iterator rend() const noexcept;
const_iterator crend() const noexcept;

Returns: frames.crend().

explicit operator bool() const noexcept;

Returns: !frames.empty().

[[nodiscard]] bool empty() const noexcept;

Returns: frames.empty().

size_type size() const noexcept;

Returns: frames.size().

size_type max_size() const noexcept;

Returns: frames.max_size().

const_reference operator[](size_type frame_no) const;

Expects: frame_no < size().

Returns: frames[frame_no].

Throws: Nothing.

const_reference at(size_type frame_no) const;

Throws: out_of_range if frame_no >= size().

Returns: frames[frame_no].

23.?.4.3 Comparisons

[stacktrace.basic.comp]

template <class Allocator2>
strong_ordering operator<=>(const basic_stacktrace< Allocator2 >& rhs) const noexcept;

Returns: this->size() <=> rhs.size() if this->size() != rhs.size(). lexicographical_compare_3way(this->begin(), this->end(), rhs.begin(), rhs.end()) otherwise.

23.?.4.4 Modifiers

[stacktrace.basic.mod]

void swap(basic_stacktrace& other);

Effects: Exchanges the contents of *this and other.

23.?.5 Non-member functions

[stacktrace.nonmembers]

void swap(stack_frame& a, stack_frame& b) noexcept;

Effects: Equivalent to a.swap(b).

```
template<class Allocator>
void swap(basic_stacktrace<Allocator>& a, basic_stacktrace<Allocator>& b);
```

Effects: Equivalent to `a.swap(b)`.

```
template<class Allocator>
string to_string(const basic_stacktrace<Allocator>& st);
```

Returns: A multiline string with a description of a stacktrace.

```
string to_string(const stack_frame& f);
```

Returns: A string with a description of `f`.

[Note: The description should provide information about contained evaluation, including information from `source_file()` and `source_line()`. - end note]

```
template<class charT, class traits, class Allocator>
basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& os, const basic_stacktrace<Allocator>& st);
```

Effects: As if by `os << to_string(bt)`;

Returns: `os`.

```
template<class charT, class traits>
basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& os, const stack_frame& f);
```

Effects: As if by `os << to_string(f)`;

Returns: `os`.

23.7.6 Hash support

[stacktrace.hash]

```
template<> struct hash<stack_frame>;
template<class Allocator> struct hash<basic_stacktrace<Allocator>>;
```

The specialization is enabled (23.14.15).

Feature-testing macro

Add a row into the "Standard library feature-test macros" table [support.limits.general]:

<code>__cpp_lib_stacktrace</code>	201811	<code><stacktrace></code>
-----------------------------------	--------	---------------------------------

VI. Acknowledgements

Many thanks to Jens Maurer, JF Bastien and Marshall Clow for pointing out many issues in the early wordings.

Special thanks to Jens Maurer for doing the core wordings.

Many many thanks to all the people who participated in the LWG meeting on 20th of August and reviewed early version of the wording.